TIP You can also use `DataAnnotations` attributes to decorate your entity classes, controlling things like column naming and `string` length. EF Core will use these attributes to override the default conventions.

As well as defining the entities, you define the `DbContext` for your application. The `DbContext` is the heart of EF Core in your application, used for all your database calls. Create a custom `DbContext`, in this case called `AppDbContext`, and derive from the `DbContext` base class, as shown in listing 12.3. This class exposes the `DbSet<Recipe>` so that EF Core can discover and map the `Recipe` entity. You can expose multiple instances of `DbSet<>` in this way for each of the top-level entities in your application.

Listing 12.3 Defining the application `DbContext`

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)    ❶
        : base(options) { }                                        ❶
    public DbSet<Recipe> Recipes { get; set; }                     ❷
}
```

❶ The constructor options object, containing details such as the connection string

❷ You'll use the Recipes property to query the database.

The `AppDbContext` for your app is simple, containing a list of your root entities, but you can do a lot more with it in a more complex application. If you wanted to, you could customize how EF Core maps entities to the database, but for this app you're going to use the defaults.

NOTE You didn't list `Ingredient` on `AppDbContext`, but EF Core models it correctly as it's exposed on the `Recipe`. You can still access the `Ingredient` objects in the database, but you must navigate